

02 异常处理与自定义异常

目标：会判断异常执行顺序，会写 `try/except/else/finally`、`raise` 和自定义异常。

异常是什么【必须闭卷会写】

1. 会出现什么问题

程序运行时可能遇到无法继续的情况，例如用户输入非数字、文件不存在、除数为 0。

2. 三类错误

类型	说明
语法错误	代码写法不合法，运行前就失败
运行时异常	运行中出错，可用异常处理
逻辑错误	程序能跑但结果错

异常处理主要处理运行时异常。

常见异常类型【必须会读会改】

```
int("abc")           # ValueError
"a" + 3               # TypeError
10 / 0                # ZeroDivisionError
[1, 2][5]             # IndexError
{"a": 1}["b"]         # KeyError
open("none.txt")      # FileNotFoundError
"abc".append("d")     # AttributeError
```

`PermissionError`：权限不足时可能出现，例如无权写入某个目录。

基础 `try/except` 【必须闭卷会写】

1. 会出现什么问题

用户输入错误会让程序崩溃。

2. 最小语法

```
try:
    score = int(input("成绩: "))
except ValueError:
    print("成绩必须是整数")
```

3. 执行顺序

```
try 没报错 → 跳过 except
try 报对应异常 → 执行 except
```

4. 正常情况

输入 "80"，转换成功。

5. 异常情况

输入 "abc"，进入 `except ValueError`。

6. 常见错误

`try` 里放太多无关代码，导致本不该捕获的错误也被捕获。

7. 考试可能怎么出

程序输出题：判断 `except` 是否执行。

多个 `except` 【必须闭卷会写】

```
try:
    number = int(input("请输入数字: "))
    result = 100 / number
except ValueError:
    print("请输入整数")
except ZeroDivisionError:
    print("不能输入 0")
```

重点：

具体异常通常写在前面
一个异常只会进入一个匹配分支

捕获异常对象：

```
except ValueError as error:  
    print(error)
```

多异常合并：

```
except (ValueError, TypeError):  
    print("类型或值错误")
```

else 【必须闭卷会写】

1. 会出现什么问题

成功后的代码如果也放在 `try` 里，可能误捕获其他错误。

2. 最小语法

```
try:  
    score = int("80")  
except ValueError:  
    print("错误")  
else:  
    print(score)
```

3. 执行顺序

只有 `try` 没有异常时执行 `else`

finally 【必须闭卷会写】

1. 会出现什么问题

有些清理动作无论成功失败都要执行。

2. 最小语法

```
try:
    print("try")
except ValueError:
    print("except")
finally:
    print("finally")
```

3. 执行顺序

无论是否异常都会执行 `finally`
即使前面 `return`，也会先执行 `finally`

现代文件处理优先使用 `with open`，不是手写 `finally: file.close()`。

完整执行顺序【必须闭卷会写】

无异常: `try` → `else` → `finally`
有匹配异常: `try` → `except` → `finally`
异常未捕获: `try` → `finally` → 异常继续向外传递

示例：

```
try:
    n = int("abc")
except ValueError:
    print("error")
else:
    print("ok")
finally:
    print("end")
```

输出：

```
error
end
```

异常与循环输入【必须闭卷会写】

```
def input_score():
    while True:
        try:
            score = int(input("成绩: "))
        except ValueError:
            print("请输入整数")
            continue

        if 0 <= score <= 100:
            return score

    print("成绩必须在 0 到 100 之间")
```

说明：

类型错误由 `except` 处理
范围错误由 `if` 处理

raise【必须闭卷会写】

1. 会出现什么问题

函数发现参数不合法时，不能只打印一句话继续跑，应主动抛异常。

2. 最小语法

```
def set_score(score):
    if not 0 <= score <= 100:
        raise ValueError("成绩必须在 0 到 100 之间")
```

3. 执行顺序

`raise` 主动抛出异常
当前正常流程中断
调用者可以 `try/except` 捕获

4. 异常传递

```
def parse_score(text):  
    return int(text)  
  
def main():  
    try:  
        score = parse_score(input("成绩: "))  
    except ValueError:  
        print("成绩格式错误")
```

`parse_score` 没处理异常，异常会传到 `main`。

5. 重新抛出【必须会读会改】

```
try:  
    int("abc")  
except ValueError:  
    print("记录错误")  
    raise
```

自定义异常【必须闭卷会写】

1. 会出现什么问题

所有错误都用 `Exception` 不清楚，调用者很难区分是哪种业务错误。

2. 最小语法

```
class InvalidScoreError(Exception):  
    pass
```

使用：

```
def validate_score(score):  
    if not 0 <= score <= 100:  
        raise InvalidScoreError("成绩必须在 0 到 100 之间")
```

捕获：

```
try:
    validate_score(120)
except InvalidScoreError as error:
    print(error)
```

3. 考试可能怎么出

要求定义一个异常类，遇到非法成绩主动抛出并捕获。

assert 【了解即可】

```
score = 80
assert score >= 0
```

assert 主要用于开发检查，不能代替正式用户输入校验。

异常和面向对象结合 【必须会读会改】

```
class InsufficientBalanceError(Exception):
    pass

class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientBalanceError("余额不足")
        self.balance -= amount
```

异常类型速查

异常	常见原因
ValueError	值不合法，如 <code>int("abc")</code>

异常	常见原因
<code>TypeError</code>	类型不匹配
<code>ZeroDivisionError</code>	除以 0
<code>IndexError</code>	索引越界
<code>KeyError</code>	字典 key 不存在
<code>FileNotFoundError</code>	文件不存在
<code>PermissionError</code>	权限不足
<code>AttributeError</code>	属性或方法不存在

完整结构执行图

```
try
├ 成功 → else → finally
├ 匹配异常 → except → finally
└ 未匹配异常 → finally → 继续向外抛
```

raise 与 except 关系图

```
函数内部 raise
→ 当前函数中断
→ 调用者 try
→ 匹配 except 捕获
```

12 个易错点

- 1. `except` 后异常类型写错。
- 2. 裸 `except`：捕获太宽。
- 3. `try` 放太多无关代码。
- 4. `else` 是无异常才执行。
- 5. `finally` 总会执行。
- 6. `finally` 会在 `return` 前执行。
- 7. `raise` 后正常流程中断。
- 8. 自定义异常要继承 `Exception`。
- 9. `assert` 不适合用户输入校验。
- 10. `except Exception` 不要随便吞掉真正 bug。
- 11. 文件不存在是 `FileNotFoundError`。
- 12. 类型转换失败是 `ValueError`。

10 个口头自测

1. `try` 没异常时 `except` 执行吗？
2. `else` 什么时候执行？
3. `finally` 什么时候执行？
4. `raise` 和自然报错有什么区别？
5. 自定义异常为什么继承 `Exception`？
6. 异常传递是什么意思？
7. `except Exception as error` 的 `error` 是什么？
8. 为什么不推荐裸 `except`？
9. `assert` 能不能做用户输入校验？
10. 文件不存在应该捕获什么异常？

6 个闭卷异常题

1. 安全整数输入。
2. 安全除法。
3. 成绩范围校验，非法则 `raise ValueError`。
4. 自定义 `InvalidScoreError`。
5. 读取文件不存在时提示。
6. 银行账户余额不足异常。

答案区

```
class InvalidScoreError(Exception):  
    pass  
  
def validate_score(score):  
    if not 0 <= score <= 100:  
        raise InvalidScoreError("成绩必须在 0 到 100 之间")  
    return score
```

[进入文件异常闭卷训练](#)